PROGRAM 1

Understanding the Background and Driving Forces for taking an Agile Approach to Software Development.

Aim:

To understand the Background and Driving Forces for taking an Agile Approach to Software Development.

Objective:

To study and understand the reasons behind the evolution of Agile methodology and to identify the key factors that make Agile suitable for modern software development.

Problem Statement:

Traditional software development models like Waterfall fail to adapt to changing requirements and result in delayed feedback and low customer satisfaction. Agile methodology was introduced to overcome these issues by promoting flexibility, collaboration and continuous delivery.

Description:

Agile software development is a modern approach that focuses on flexibility, collaboration and delivering working software quickly and continuously. It emerged as a response to the problems and limitations faced in traditional software development methods like the Waterfall model.

Background of Agile Approach

It is not possible to get all customer requirements at the beginning of the project. Requirements can get changed or modified. There is always scope for more than what time and money allow.

1. **Problems in Traditional Models:**
    - In traditional methods (like Waterfall), each phase (requirements, design, coding, testing) is done one after the other.
    - Changes are difficult once development starts.
    - Testing is done at the end, so issues are discovered late.
    - Customers have to wait till the end to see the product.
    - Final product may not meet customer expectations if requirements change.
2. **Need for a Flexible Model:**
    - Software requirements often change during the development process.
    - Customers need to see progress regularly and give feedback.
    - Fast delivery of working software is necessary in the competitive IT world.

3. **Introduction of Agile:**
    - In 2001, 17 software experts created the Agile Manifesto, which gave the principles of Agile development.
    - The focus shifted to people, collaboration, quick delivery and responding to change.

    It is focusing on:
    - Individuals and interactions over processes and tools.
    - Working software over comprehensive documentation.
    - Customer collaboration over contract negotiation.
    - Responding to change over following a fixed plan.

Driving Forces of Agile Development

Driving forces are the real-world problems, challenges, goals and changing needs in software development that forced people to find a better way that led people to adopt Agile. The main reasons that pushed the software industry to move from traditional development models to Agile methods are listed below.

1. **Customer Satisfaction:**
    - Agile gives importance to the customer.
    - Software is delivered in small parts (called iterations or sprints) and the customer can see working results early.
2. **Flexibility and Adaptability:**
    - Requirements can change even late in development.
    - Agile teams are ready to make changes and adjust their plans.
3. **Continuous Delivery:**
    - Agile promotes delivering small and usable pieces of software frequently.
    - This helps in faster releases and quicker feedback.
4. **Better Collaboration and Communication:**
    - Agile encourages close interaction between developers, testers and customers.
    - Daily meetings (called stand-ups) help the team stay in sync.
5. **Early Detection of Errors:**
    - Testing is done throughout the development cycle.
    - Problems are detected early, reducing the cost and time of fixing bugs.
6. **Team Empowerment and Motivation:**
    - Agile teams are self-organized and take ownership of their work.
    - This increases motivation and productivity.

7. **Improved Quality:**
    - Regular testing, continuous integration and feedback ensure better software quality.

Conclusion:

This Agile approach to software development is introduced to overcome the disadvantages of traditional model like Waterfall approach. The Agile practices enhance the collaboration, adaptability, flexibility and delivering working software quickly and continuously.

PROGRAM 2

Understand the business value of adopting agile approach.

Aim:

To study and understand how the Agile methodology contributes to business value in terms of speed, flexibility, customer satisfaction and team productivity in software development.

Objective:

1. To understand how Agile methods reduce risk and support cost-effective development.
2. To compare Agile with traditional software development models in terms of business impact.
3. To identify the core principles of Agile that support business goals.

Problem statement:

In traditional software development models like the Waterfall approach, late delivery, rigid requirements and lack of customer involvement often led to project failure or dissatisfaction. In today's competitive and fast-evolving business environment, organizations require a methodology that supports faster delivery, frequent updates, active customer involvement to adapt to change. This experiment aims to understand how Agile methodology addresses these problems and delivers measurable business value by improving delivery speed, product quality and customer satisfaction through its iterative and adaptive approach.

Description:

Agile is a modern, iterative software development approach that emphasizes collaboration, flexibility, continuous feedback and high-quality outputs. Agile frameworks (like Scrum, Kanban, XP) help organizations respond to change faster, reduce time to market and align better with customer needs. Traditional models like Waterfall often fail to adapt to changes once the project has started, while Agile welcomes changes at any stage of development.

Business Value of Adopting Agile Approach:

Business value is strongly related to the organizations ability to make fast business decisions based on market changes, customers and business trends and pivot when needed. Business value refers to the clear and conceptual benefits an organization gain, such as:

1. **Faster time to market**:

   Iterative releases (sprints) mean that usable features are delivered quickly. Businesses gain competitive advantage by releasing before their competitors.

   Example: Develop a food delivery app.

The Agile team breaks down the project into features and starts with core features:

- User login
- Restaurant list
- Place order

These are delivered in Sprint 1, allowing the company to launch a minimum viable product (MVP) in just 2 weeks.

Business Value: Company starts getting users and market feedback early.

2. **Customer satisfaction**:

   Agile focuses on regular customer feedback, ensuring the product meets actual user needs and increasing customer satisfaction. The real-time changes can be incorporated quickly based on stakeholder feedback.

   Example: After Sprint 1, customers complain that the order tracking feature is missing. The team adds "Live Order Tracking" in Sprint 2 based on feedback.

   Business Value: Users feel heard, leading to better retention and satisfaction.

3. **Better product quality:**

   Frequent testing and continuous integration improve product reliability. The changes in requirements are welcomed, improving adaptability in dynamic markets.

   Example: The team uses Test-Driven Development (TDD) and performs code reviews every sprint.

   Business Value: High-quality product with fewer post-launch issues.

4. **Reduced risks and cost**: Short development cycles reduce the risk of complete failure. Problems are identified early, reducing the cost and impact of errors. Agile minimizes wastage by focusing only on essential features. Early delivery enables early revenue generation.

   Example: A bug is found in payment gateway integration during Sprint 2. The Agile process ensures frequent testing, so the bug is caught early and fixed before going live. Initially, the company thought to build a full app in 6 months. But by Agile delivery, they launched a basic version in 4 weeks, saving time and money.

   Business Value: Avoids financial loss or reputation damage, reduced development cost, efficient resource use.

5. **Adaptability to Change:** The ability of a software development team to respond quickly and efficiently to new or changing requirements, customer feedback, or market trends at any stage of the development process.

Example: Midway, a competitor app launches a "wallet cashback" feature. The Product Owner adds this as a new backlog item and the team implements it in Sprint 3.

Business Value: Business stays competitive and aligned with market trends.

6. **Improved team collaboration**: Daily stand-up meetings, retrospectives and shared ownership boost motivation and productivity. Stakeholders have visibility into every stage of the project.

Conclusion:

The Agile approach significantly enhances business value by delivering faster, reducing waste, improving quality and engaging stakeholders throughout the project lifecycle.

PROGRAM 3

Understand agile development practices.

Aim:

Implementation of Agile Development Practices.

Objective:

- To gain a practical understanding of Agile software development methodologies.
- To simulate an Agile development process (e.g., Scrum or Kanban).
- To experience iterative development, sprint planning, daily stand-ups and retrospective.

Problem Statement:

To simulate an Agile (Scrum/Kanban) workflow to develop a small software project (e.g., a To-Do List App) and analyze its effectiveness compared to traditional approaches.

Description:

1. Agile Methodology

Agile is an iterative and incremental approach to software development that focuses on collaboration, customer feedback and small, rapid releases. Key principles include:

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

2. Common Agile Frameworks

a) Scrum

- **Roles:** Product Owner, Scrum Master, Development Team.
- **Artifacts:** Product Backlog, Sprint Backlog, Increment.
- **Ceremonies:**
  - **Sprint Planning** (selecting tasks for the sprint).
  - **Daily Stand-up** (15-minute progress check).
  - **Sprint Review** (demo to stakeholders).
  - **Sprint Retrospective** (team improvement discussion).

b) Kanban- Focuses on visualizing work, limiting work in progress (WIP) and continuous delivery.

- Visual workflow (Kanban board with columns: To Do, In Progress, Done).
- Work In Progress (WIP) limits (prevents overloading the team).
- Continuous delivery (no fixed sprints, tasks pulled as capacity allows).

3. Requirements

3.1 Software/Hardware

- Project Management Tool: Trello, Jira, or physical Kanban board.
- Development Environment: VS Code, Git, Python/Java/JavaScript(based on project).
- Collaboration Tools: Microsoft Teams for communication.

3.2 Team Structure (For Scrum Simulation)

- Product Owner (PO): Defines requirements (user stories).
- Scrum Master (SM): Ensures Agile practices are followed.
- Developers (3-5 members): Code, test and deliver features.

4. Procedure

4.1 Setup

1. Form a team (Product Owner, Scrum Master, Developers).
2. Choose a project (e.g., a basic web/mobile app).
3. Set up a task board (To Do, In Progress, Done).

4.2 Sprint planning

1. Define user stories (e.g., "As a user, I want to add tasks to my to-do list").
2. Prioritize tasks in the product backlog.
3. Select tasks for Sprint 1 (e.g., UI design, basic functionality).

4.3 Daily Stand-ups (15 mins each day)

Each member answers:

- What did you do yesterday?
- What will you do today?
- Any blockers?

4.4 Development & Testing

- Developers code features, update the Kanban board.
- Continuous Integration (CI) checks (if automated testing is set up).

4.5 Sprint Review & Retrospective

1. Give a demo working software to stakeholders.
2. Discuss the improvements (What went well? What didn't?)

4.6 Sprint 2 (Enhancements & Feedback Incorporation)

- Add new features (e.g., task categories, due dates).
- Refactor code based on feedback.

5. Observations

Agile Metrics Tracking-

Metric	**Sprint 1**	**Sprint 2**
Tasks Completed	5/8	7/8
Bugs Reported	3	1
Customer Feedback	Basic UI works	More features needed

Conclusion:

The above experiment successfully simulated Agile development, experiencing iterative progress, teamwork and adaptive planning. Agile practices improve flexibility and customer satisfaction compared to traditional models like Waterfall.

PROGRAM 4

Drive development with Unit Test using Test Driven Development.

Aim:

To implement the Test-Driven Development (TDD) for a Software Module using Unit Testing.

Objective:

To understand and implement the concept of Test-Driven Development (TDD) by writing unit tests before developing the actual code, thus ensuring software quality and reducing bugs early in the development process.

Problem Statement:

Develop a simple calculator application that performs basic arithmetic operations such as addition, subtraction, multiplication and division. Implement the functionality by writing test cases first using a unit testing framework (like unittest in Python or JUnit in Java), following the principles of TDD.

Description:

Test-Driven Development (TDD)

TDD is a software development approach where:

1. Tests are written before the actual code.
2. Code is written only to pass the failing tests.
3. The cycle repeats with refactoring for optimization.

Test Driven Development (TDD) is a software development process where test cases are written before writing the actual code. The typical TDD cycle follows these steps:

- **Red**: Write a failing test case for the desired functionality.
- **Green**: Write the minimum amount of code to make the test pass.
- **Refactor**: Improve the code structure without changing its functionality

Procedure:

1. Identify a small piece of functionality to implement (e.g., addition).
2. Write a test case that defines the expected behavior for that functionality.
3. Run the test case – it should fail (Red).
4. Write just enough code to make the test pass (Green).
5. Run all tests to ensure they pass.
6. Refactor the code to improve structure while keeping the tests green.
7. Repeat steps 1–6 for the remaining features.

Step-by-Step TDD Implementation

Step 1: Setup Testing Environment

Install Python Extension in VS Code

Make sure you have:

- Python installed on your machine
- Python extension installed in VS Code

Step 2: Write a Failing Test (Red Phase)

```
# test_calculator.py
import unittest
from calculator import Calculator
def test_add():
    calc = Calculator()
    assert calc.add(2, 3) == 5 # Expected to fail (Calculator not implemented yet)
```

Step 3: Implement Minimal Code to Pass Test (Green Phase)

```
# calculator.py
class Calculator:
    def add(self, a, b):
        return a + b # Just enough to pass the test
```

Step 4: Run Tests

```
pytest test_calculator.py -v
```

Step 5: Repeat for other methods

- Write failing test → Implement minimal code → Refactor.
- Example for divide() with error handling:

```
# test_calculator.py
def test_divide_by_zero():
    calc = Calculator()
    with unittest.raises(ValueError):
        calc.divide(5, 0)
```

Source code:

1. The **module code** (calculator.py) – where we define the functions

   CODE:

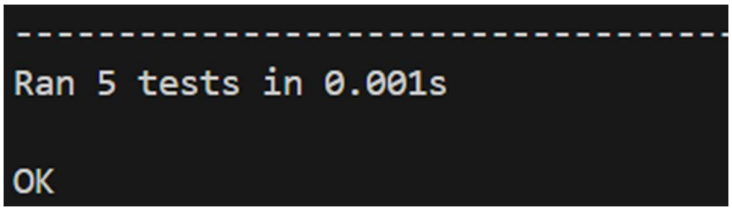
```
# calculator.py
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        raise ValueError("Division by zero is not allowed")
    return a / b
```

2. The **test code** (test_calculator.py) where we write unit tests before actual implementation (as per TDD).

```
# test_calculator.py
import unittest
import calculator
class TestCalculator(unittest.TestCase):
    def test_add(self):
        self.assertEqual(calculator.add(2, 3), 5)
        self.assertEqual(calculator.add(-1, 1), 0)
        self.assertEqual(calculator.add(0, 0), 0)
    def test_subtract(self):
        self.assertEqual(calculator.subtract(10, 5), 5)
        self.assertEqual(calculator.subtract(0, 7), -7)
        self.assertEqual(calculator.subtract(7, 0), 7)
    def test_multiply(self):
        self.assertEqual(calculator.multiply(3, 4), 12)
        self.assertEqual(calculator.multiply(-2, 5), -10)
        self.assertEqual(calculator.multiply(0, 5), 0)
    def test_divide(self):
```

```
        self.assertEqual(calculator.divide(10, 2), 5)
        self.assertEqual(calculator.divide(9, 3), 3)
        self.assertAlmostEqual(calculator.divide(1, 3), 0.3333333, places=6)
    def test_divide_by_zero(self):
        with self.assertRaises(ValueError):
            calculator.divide(10, 0)
if __name__ == '__main__':
    unittest.main()
```

Output:



Conclusion:

The above program demonstrated the practice of Test-Driven Development where test cases were written before the implementation of logic. This approach led to better design, fewer bugs and a more reliable codebase. The unit test cases were successfully implemented using the TDD approach. All tests passed, validating that the calculator functions work as intended.

PROGRAM 5

Apply Design principle and Refactoring to achieve agility.

Aim:

To implement a software solution that demonstrates the application of design principles and refactoring techniques in order to improve code quality, maintainability and agility in software development.

Objective:

The objective of this experiment is to understand how applying software design principles (such as SOLID, DRY and OCP) along with refactoring techniques improves the flexibility, maintainability and readability of code.

Problem Statement:

In agile development, code evolves over time due to changing requirements. Without proper design principles and refactoring, the codebase may become rigid, difficult to maintain and prone to bugs. The task is to write a program that initially has poor design and then apply design principles and refactoring to make the program more flexible, maintainable and adaptable to future changes.

Description:

1. Design Principles for Agility

Design principles are fundamental guidelines that help in writing clean, maintainable and scalable code. Common principles include:

Open/Closed Principle:

Design modules to be open for extension but closed for modification, enabling new features without changing existing code.

Single Responsibility Principle (SRP):

In states that a class or module should have only one reason to change. This means it should have only one responsibility or purpose.

Encapsulation:

It hide the internal details using classes or functions. Expose only what is necessary. This enables changes without affecting other components.

DRY (Don't Repeat Yourself):

It is used to eliminate duplicate code by using functions, classes, or reusable modules. Updates only need to be made in one place.

These principles reduce coupling, improve code readability and support agility.

2. Refactoring Techniques

Refactoring is the process of restructuring existing code without changing its external behavior.

Purpose: It improves design, remove code smells, enhance readability and make future changes easier.

Common Techniques:

Rename for Clarity: It improve variable, method and class names.

E**xtract Method:** It breaks large methods into smaller, reusable functions.

Inline Temporary Variables: It eliminates redundant variables.

Remove Dead Code: Clear unused functions or blocks.

Replace Magic Numbers with Constants: Use meaningful constant names.

By applying design principles and refactoring, we ensure that the code remains agile, that means it can adapt to new requirements quickly without major rewrites.

Algorithm

1. **Identify Poorly Designed Code:**
    - Write a simple program that violates SRP, OCP and other principles (e.g., one class doing too many tasks).
2. **Analyze Code Smells:**
    - Look for long methods, duplicate code, high coupling, low cohesion or hard-coded values.
3. **Apply Refactoring Steps:**
    - Break large classes into smaller ones (SRP).
    - Introduce interfaces or abstract classes for extension (OCP).
    - Remove duplicated logic by extracting methods.
    - Rename variables/methods to be meaningful.
4. **Verify Behavior:**
    - Test the program after each refactoring to ensure output remains the same.
5. **Document Changes:**
    - Record initial problems and improvements after applying design principles.

Source code:

bad_calculator.py

```
class Calculator:
    def calculate(self, a, b, operation):
        if operation == "add":
            return a + b
        elif operation == "subtract":
            return a - b
        elif operation == "multiply":
            return a * b
        elif operation == "divide":
            if b == 0:
                raise ValueError("Division by zero!")
            return a / b
        else:
            raise ValueError("Invalid operation")
# Example usage
if __name__ == "__main__":
    calc = Calculator()
    print("10 + 5 =", calc.calculate(10, 5, "add"))
    print("10 - 5 =", calc.calculate(10, 5, "subtract"))
    print("10 * 5 =", calc.calculate(10, 5, "multiply"))
    print("10 / 5 =", calc.calculate(10, 5, "divide"))
```

Output (Before Refactoring)

```
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2.0
```

refactored_calculator.py

```
from abc import ABC, abstractmethod
# Base abstract class (Abstraction)
class Operation(ABC):
    @abstractmethod
    def execute(self, a, b):
        pass
# Individual operation classes (SRP applied)
class Add(Operation):
    def execute(self, a, b):
        return a + b
class Subtract(Operation):
    def execute(self, a, b):
        return a - b
class Multiply(Operation):
    def execute(self, a, b):
        return a * b
class Divide(Operation):
    def execute(self, a, b):
        if b == 0:
            raise ValueError("Division by zero!")
        return a / b
# Calculator uses composition (OCP applied)
class Calculator:
    def __init__(self, operation: Operation):
        self.operation = operation
     def calculate(self, a, b):
        return self.operation.execute(a, b)
```

```
# Example usage
if __name__ == "__main__":
    add_calc = Calculator(Add())
    print("10 + 5 =", add_calc.calculate(10, 5))
    sub_calc = Calculator(Subtract())
    print("10 - 5 =", sub_calc.calculate(10, 5))
    mul_calc = Calculator(Multiply())
    print("10 * 5 =", mul_calc.calculate(10, 5))
    div_calc = Calculator(Divide())
    print("10 / 5 =", div_calc.calculate(10, 5))
```

Output (After Refactoring)

```
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2.0
```

Conclusion:

The above program concluded that applying design principles ensures cleaner and more modular code, while refactoring enhances the internal structure without changing functionality. Together, they make the codebase more adaptable and improve testability. This promotes agile practices by enabling quick responses to change and deliver of high-quality software within shorter iterations.

PROGRAM 6

To Study Automated Build Tool.

Aim:

To study and understand the concept, working and benefits of automated build tools used in software development for compiling, testing and deploying applications automatically.

Objective:

To understand the concept, features and usage of automated build tool and their role in software development.

Problem Statement:

In modern software development, projects often involve multiple source files, libraries, dependencies and repetitive tasks such as compilation, packaging, testing and deployment. Doing these tasks manually is error-prone, time-consuming and inconsistent. An automated build tool simplifies the process by automating these tasks and ensuring consistent builds.

Description:

1. Introduction to Automated Build Tools

An automated build tool is software that automates the process of converting source code into executable applications.

It handles tasks such as:

- Compiling source code
- Managing dependencies
- Running automated tests
- Packaging and deploying applications
- Generating documentation

This automation ensures faster development, fewer errors and better consistency in builds.

2. Commonly Used Automated Build Tools

- **Maven** – Java-based project build & dependency management tool.
- **tox** – Automates testing in multiple environments.
- **invoke** – Task execution tool (similar to Make).
- **nox** – Similar to tox, but more flexible.
- **Poetry** – Dependency management and packaging tool.
- **Gradle** – Flexible build automation tool supporting multiple languages.
- **Ant** – Java-based build tool with XML configuration.
- **Make** – Build automation tool for C/C++ projects.
- **NPM Scripts** – For JavaScript/Node.js projects.

3. Features of Automated Build Tools

1. **Dependency Management** – It automatically downloads required libraries.
2. **Task Automation** – It builds, tests and packages code without manual steps.
3. **Incremental Builds** – It only rebuilds changed files to save time.
4. **Integration with CI/CD** – Works with Jenkins, GitHub Actions, etc.
5. **Cross-Platform Support** – It can run on Windows, Linux, Mac.

Procedure (Example using **tox)**

1. Install Python and pip.
2. Install tox:

   pip install tox
3. Create a sample Python project with:
   - hello.py (main script)
   - test_hello.py (test file)
4. Create a tox.ini configuration file to define environments and test commands.
5. Run:

   tox
6. Observe automated environment creation, dependency installation and test execution.

Example Implementation using tox

File: hello.py

```
def greet(name): return f"Hello, {name}!"
```

File: test_hello.py

```
from hello import greet
 def test_greet(): assert greet("Python") == "Hello, Python!"
```

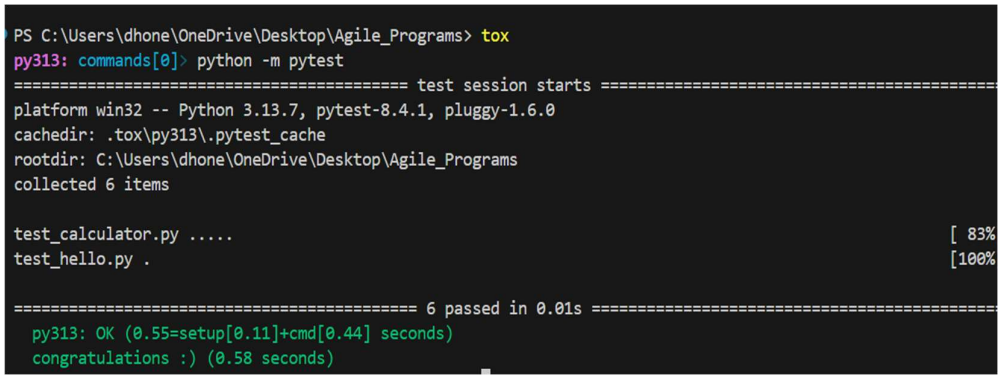
File: tox.ini

```
[tox]
envlist = py313
[testenv]
deps =
   pytest
commands =
   python -m pytest
```

Steps to Run

1. Install tox:
   pip install tox
2. Run build and test:
   tox

Output:



Conclusion:

Automated build tools in Python like tox, nox and poetry streamline development by automating testing, dependency management and packaging. They improve productivity, ensure consistency across environments and are essential in agile and CI/CD workflows.

PROGRAM 7

To study Version Control Tool.

Aim:

To study and understand the tracking, managing and control changes to code by using version control tools.

Objective:

To understand the concept, features and usage of Version Control Tools (such as Git) and their role in software development.

Problem statement:

In software development, maintaining and tracking different versions of code manually is difficult and error-prone. Collaboration among multiple developers often leads to conflicts and loss of updates. To solve this, version control tools like Git are used to manage versions, ensure collaboration and provide backup.

Description:

Version Control Tools are software systems that help track, manage and control changes to code or documents. They enable multiple developers to collaborate effectively by maintaining a history of changes, allowing rollback to previous versions and resolving conflicts.

Types:

1. **Centralized Version Control (CVCS):** Single server stores all versions (e.g., Subversion, CVS).
2. **Distributed Version Control (DVCS):** Each developer has a local copy of the repository (e.g., Git, Mercurial).

Features of Version Control Tools:

- Track history of changes.
- Branching and merging support.
- Collaboration among team members.
- Backup and recovery of code.
- Conflict resolution mechanisms.

Procedure:

1. Install a version control tool (**e.g., Git**).
2. Configure Git with username and email.
3. Create a new repository (**git init**).
4. Add files to the repository (**git add**).

5. Commit changes with a message (**git commit**).
6. Check history using (**git log**).
7. Create and switch branches (**git branch, git checkout**).
8. Merge branches (**git merge**).
9. Push and pull changes to/from a remote repository (**GitHub**).

Observation:

- Repository successfully initialized.
- Changes tracked and stored in commit history.
- Branches created and merged without data loss.
- Remote repository synchronization achieved.

OUTPUT:

1. To verify git installation: **Git --version**
2. Config git
    a. **git config --global user.name "namexyz"**
    b. **git config --global user.email "emailxyz"**
3. To verify the configuration: **git config --list**
4. Now create or open a project folder
5. Initialize git repository: **git init**
6. Create a sample file in the project folder
7. Check git status**: git status**
8. Add file to staging area: **git add filename**
9. Commit changes: **git commit -m filename**
10. For connecting git to github
    git remote add origin (repo ink)
    git remote add origin https:/github.com/yourusername/github repo name.git
11. Push code to GitHub
    git branch -m master
    git push -u origin master

Conclusion:

The study of version control tools (Git) demonstrated how developers can efficiently track changes, collaborate in teams and maintain different versions of a project. It ensures better software quality, reduces conflicts and provides backup for the codebase.

PROGRAM 8
To study Continuous Integration tool.

Aim:

To study and understand the working of a Continuous Integration (CI) tool (e.g., Jenkins/GitLab CI/GitHub Actions) for automating the building, testing and integration of code in a software development environment.

Objectives:

1. To learn the concept of Continuous Integration and its importance in software development.
2. To set up and configure a CI tool to automate builds and testing.
3. To understand the pipeline stages – build, test, deploy.
4. To analyze the results and logs generated by the CI tool.

Problem Statement:

Manual integration of code from multiple developers often leads to conflicts, build failures, and delays in software delivery. A CI tool is needed to automatically integrate code changes, run tests and generate build artifacts to improve software quality and development speed.

Description:

Continuous Integration (CI) is a DevOps practice where code changes from all developers are automatically integrated into a shared repository several times a day. Each integration triggers an automated build and test process to detect integration issues early.

A CI tool (such as Jenkins) can:

- Pull code automatically from version control (like GitHub).
- Build the code into executables or deployable packages.
- Run automated tests to ensure code quality.
- Provide feedback to developers immediately.

This experiment involves installing or accessing a CI tool, connecting it to a version control repository, configuring a build pipeline and observing the automation of the build/test process.

Procedure:

1. **Install/Access CI Tool:** Install Jenkins (or use a cloud-based CI tool like GitHub Actions).
2. **Connect to Repository:** Link the CI tool to a version control system (GitHub/GitLab).
3. **Create a New Project/Pipeline:** In the CI dashboard, create a new job or pipeline.

4. **Configure Build Steps:**
    - Specify repository URL.
    - Define build commands (compile/test).
5. **Set Triggers:** Enable automatic builds on each code commit or scheduled builds.
6. **Run the Pipeline:** Commit a change to the repository to trigger the pipeline.
7. **Monitor Build and Test Results:** View logs, test reports, and artifacts.
8. **Analyze Outcomes:** Check if the build succeeded and tests passed; troubleshoot if errors occur.

Output:

- git clone <repo link>
- cd repo name
- mkdir -p .github/workflows

create a github action workflow folder

- code .github/workflows/deploy.yml

Add workflow content - add code into deploy.yml file

- git add filename
- git commit -m “messaage”
- git push origin master
- check github dashboard to check workflow at actions tab.
- then deploy code file running automatically.

Conclusion:

The experiment successfully demonstrated the use of a Continuous Integration tool to automate building and testing of software whenever code changes occur. CI improves collaboration, detects errors early and increases the overall efficiency of the software development process.

PROGRAM 9

Perform Testing Activities within an Agile Project.

Aim:

To study and perform testing activities in an Agile project environment, demonstrating how testing is integrated into each iteration/sprint.

Objective:

1. To understand how testing is embedded in Agile sprints.
2. To plan and execute test cases concurrently with development.
3. To practice writing user stories and their acceptance criteria.
4. To run manual/automated tests as part of Agile iteration.

Problem Statement:

In traditional software development, testing occurs at the end of the lifecycle, leading to late detection of defects and increased costs. Agile requires continuous testing within each sprint to ensure working, high-quality increments. This experiment focuses on implementing testing activities (planning, execution, automation) within an Agile iteration.

Description:

In Agile projects, testing is not a separate phase but an ongoing activity performed throughout the sprint. Testers, developers and product owners collaborate to define **acceptance criteria** for user stories. Testing activities include:

- Writing **test cases** alongside user stories.
- Performing **unit testing**, **integration testing** and **acceptance testing** within the sprint.
- Using automation tools (like Selenium, JUnit, Pytest) to ensure quick feedback.
- Continuous integration ensures tests run automatically on each commit.

This experiment demonstrates how to take a small user story, write its acceptance criteria, create test cases, execute them (manually or automatically) and record results within a sprint.

Procedure:

1. **Select a User Story:** Choose a small functionality (e.g., “As a user, I can log in to the application”).
2. **Define Acceptance Criteria:** Document clear conditions for the story to be considered “Done”.
3. **Write Test Cases:** Prepare test scenarios and cases based on acceptance criteria.
4. **Develop the Feature:** Developer implements the story during the sprint.
5. **Perform Unit Tests:** Run unit tests immediately after coding.

6. **Perform Integration Tests:** Verify the new functionality integrates with existing modules.
7. **Perform Acceptance Tests:** Validate story against acceptance criteria with the Product Owner.
8. **Automate Tests (Optional):** Use an automation tool (e.g., Selenium/Pytest) for repetitive checks.
9. **Log and Track Defects:** Report issues in the Agile board (Jira/Trello).
10. **Review During Sprint Review:** Demonstrate the working story with test results.

Output:

User Story: As a user, I can log in to the application.
Acceptance Criteria: Verify that all acceptance criteria are met.

User can log in with valid credentials.
Error message shown for invalid login.
Login redirects to dashboard.

Test Summary:

Unit Tests: Show that individual code components work correctly **(**15 passed / 0 failed)
Integration Tests: Confirm the new feature works well with other modules (10 passed / 0 failed).
Acceptance Tests: Verify that all acceptance criteria are met.

Defects: any bugs or issues discovered during testing are:

- Logged in tools like Jira or Trello.
- Assigned to developers for fixing.
- Tracked until resolved.

Status: Story completed and accepted in Sprint Review.

Conclusion:

Testing activities were successfully performed within the Agile project during the sprint. This demonstrated that Agile testing is continuous, collaborative, and aligned with user stories, ensuring high quality and quick feedback for each increment of software.

PROGRAM 10

Build and test applications with continuous integration

Aim:

To install Jenkins and configure it to test and deploy Java or web applications using NetBeans or Eclipse IDE through a Continuous Integration (CI) process.

Objective:

1. To understand the concept and workflow of Continuous Integration (CI).
2. To install and configure Jenkins on a local system.
3. To integrate Jenkins with Git and an IDE (NetBeans/Eclipse).
4. To automate the build, testing, and deployment of a Java or web application.

Problem Statement:

In manual deployment, developers spend significant time testing, building, and deploying code after every update. Continuous Integration tools like Jenkins automate these steps, improving efficiency and reducing human error. This experiment aims to implement CI using Jenkins for a Java or web application.

Procedure:

1. **Install Java JDK:**
   - Download and install the latest JDK.
   - Set the JAVA_HOME environment variable.
2. **Install Jenkins:**
   - Download Jenkins.
   - Run Jenkins using the command:
     java -jar jenkins.war
   - Access Jenkins in the browser at http://localhost:8080.
   - Unlock Jenkins using the initial admin password provided in the terminal.
   - Install the suggested plugins during setup.
3. **Create a Jenkins Job:**
   - Open Jenkins dashboard → New Item → Freestyle project.
   - Name the project (e.g., "WebApp-CI-Test").
   - In the project configuration, link your Git repository under source code Management.
   - Specify the build steps using invoke top-level Maven targets or execute shell commands.
4. **Build and Test Application:**
   - In Jenkins, click build now to start the job.
   - Jenkins fetches the code from the repository, compiles it, runs tests, and shows build results.
   - Observe the console output for build status (Success/Failure).

5. **Integrate with NetBeans/Eclipse (Optional):**
    - Use the Jenkins plugin available for your IDE.
    - Configure Jenkins server details inside the IDE to trigger builds automatically when code is pushed.
6. **Deployment:**
    - Configure Jenkins to deploy the application to a local Tomcat server or any web container after a successful build.

Output:

- Jenkins successfully fetches code from GitHub or local repository.
- Application build and testing are automated.
- The build status is displayed on Jenkins Dashboard (Success or Failure).
- Deployment is triggered automatically on successful builds.

Conclusion:

Continuous Integration using Jenkins automates the entire software build, test and deployment process. It ensures faster feedback on code changes, reduces manual effort and improves software reliability and development efficiency.

PROGRAM 11

Version control

Aim:

To perform version control operations such as creating repositories, committing changes, viewing history and reverting versions on a web project using **Git and Mercurial**.

Objective:

1. To understand the concept and importance of version control systems.
2. To track changes in website code and maintain multiple versions efficiently.
3. To learn how to create, commit and manage versions of website code using Mercurial.

Problem Statement:

Managing and tracking code changes manually during website development can lead to confusion and data loss. Version Control Systems (VCS) like **Mercurial and Git** help automate this process by keeping track of all versions of the source code, enabling developers to revert, merge or synchronize changes easily.

Using Mercurial

Procedure:

1. **Install Mercurial:**
    - Download and install from https://www.mercurial-scm.org.
    - Verify installation:

      hg --version
2. **Create a Web Project Folder:**
    - Create a folder named MyWebsite.
    - Add files such as:
        - index.html
        - style.css
3. **Initialize Repository**

   hg init
4. **Add Files to Repository:**

   hg add
5. **Commit the Changes:**

   hg commit -m "Initial version of website"
6. **Modify and Recommit Changes:**
    - Edit index.html (add new section or text).

- Then run:

  hg commit -m "Updated homepage content"

7. **View Repository Status and History:**
   - Check modified files:

     hg status
   - View commit history:

     hg log
8. **Create a Remote Repository:**
   - Use **Bitbucket** (supports Mercurial) or a self-hosted repository.
   - Link and push changes:

     hg push <remote_URL>

Using Git

Procedure:

1. **Install Git:**
   - Download and install Git from https://git-scm.com.
   - Verify installation using the command:

     git --version
2. **Create a Web Project Folder:**
   - Create a new folder, e.g., MyWebsite.
   - Inside it, create basic files like:
     - index.html
     - style.css
3. **Initialize Git Repository:**
   - Open the terminal in the project folder and type

     git init
   - This creates a new local repository.
4. **Add Files to Repository**

   git add filename
5. **Commit the Changes:**

   git commit -m "Initial version of website"
6. **Modify and Commit New Changes:**
   - Edit index.html (add new content).
   - Then run:

     git add index.html

```
git commit -m "Updated home page content"
```

7. **View Commit History:** git log
8. **Create a Remote Repository:**
    - Create a repository on **GitHub**.
    - Link it using:

```
git remote add origin <repo_URL>
git push -u origin main
```

Output:

- Mercurial and Git are successfully managing and tracks all file changes in the website project.
- Revision history can be viewed with commit details and authorship.
- Older versions can be restored easily.
- Source code is safely versioned and organized.

Conclusion:

Both **Git** and **Mercurial** are distributed version control systems that allow developers to track and manage code changes efficiently. Git is widely popular due to its flexibility, powerful branching and strong integration with platforms like GitHub.

PROGRAM 12

Virtualization and Containerization

Aim:

To Install and Configure Docker for Creating Containers of Different Operating System Images such as Ubuntu, CentOS and Alpine Linux.

Objective:

1. To understand the concepts of virtualization and containerization.
2. To install and configure Docker on a local or virtual machine.
3. To create, run and manage containers of various OS images.
4. To differentiate between virtual machines and containers in terms of performance and resource usage.

Problem Statement:

Traditional virtualization uses separate virtual machines that consume large system resources. This experiment aims to demonstrate how Docker enables efficient deployment of multiple operating system environments within a single host using containers.

Procedure:

1. **Install Docker:**
   - Download and install Docker from https://www.docker.com.
   - Verify installation using:

     docker --version
2. **Start Docker Service:**
   - Ensure Docker daemon is running:

     sudo systemctl start docker

     sudo systemctl enable docker
3. **Check Docker Info:**
   - To verify system configuration:

     docker info
4. **Pull Different OS Images:**
   - To download images from Docker Hub:

     docker pull ubuntu

     docker pull centos

     docker pull alpine

5. **List Downloaded Images:**

   docker images

6. **Create and Run Containers:**
   - For Ubuntu:

     docker run -it ubuntu /bin/bash
   - For CentOS:

     docker run -it centos /bin/bash
   - For Alpine:

     docker run -it alpine /bin/sh
7. **Check Running Containers:**

   docker ps
8. **List All Containers (including stopped ones):**

   docker ps -a
9. **Stop and Remove Containers:**

   docker stop <container_id>

   docker rm <container_id>
10. **Remove Unused Images (optional):**

    docker rmi <image_name>

Output:

- Docker was successfully installed and configured.
- Containers of Ubuntu, CentOS and Alpine operating systems were created and executed.
- Each container provided an isolated environment for running OS-level commands independently.

Conclusion:

Docker enables lightweight and efficient containerization, allowing multiple operating system environments to run simultaneously on a single host. Compared to virtualization, containerization consumes fewer resources, provides faster startup times and simplifies application deployment and testing.

PROGRAM 13

Virtualization and Containerization – To Build, Deploy and Manage a Web/Java Application on Docker

Aim:

To study and understand virtualization and containerization techniques by building, deploying and managing a simple Java/Web application using Docker containers.

Objectives:

1. To learn the basics of Docker images, containers, Dockerfile and Docker Hub.
2. To containerize a Java/Web application using Docker.
3. To run the application inside a Docker container.
4. To understand how Docker simplifies deployment across different environments.

Problem Statement:

Modern software development demands applications that are easily deployable, scalable and platform-independent. Traditional deployment methods cause issues like environment mismatch, dependency errors and inconsistent behavior across systems.
To solve this, Docker containerization provides a lightweight and consistent environment.

Procedure:

Step 1: Install Docker

- Install Docker Desktop (Windows/Mac) or Docker Engine (Linux).
- Verify installation:

docker --version

Step 2: Create a Simple Java Web Application

Example: Create a file Hello.java

```
public class Hello
{
    public static void main (String[] args)
        {
          System.out.println("Hello from Dockerized Java Application!");
        }
}
```

Compile and test locally

```
javac Hello.java
java Hello
```

Step 3: Create a Dockerfile

Create a file named **Dockerfile** in the same directory:

```
FROM openjdk:17
COPY Hello.java /app/Hello.java
WORKDIR /app
RUN javac Hello.java
CMD ["java", "Hello"]
```

Step 4: Build the Docker Image

Run the following command in the terminal:

docker build -t java-docker-app .

This creates an image named **java-docker-app.**

Step 5: Run the Docker Container

To run the docker container-

docker run --name mycontainer java-docker-app

Step 6: Verify Output

- The output should display in the console.
- Use Docker commands:

  List running containers: **docker ps**

  View logs: **docker logs mycontainer**

Output:

Console Output after running container:

Hello from Dockerized Java Application!

Conclusion:

The experiment demonstrated how containerization using Docker helps package a Java/Web application along with its dependencies into a portable image that runs consistently on any system. This reduces deployment issues, eliminates environment conflicts and makes application management easier.

PROGRAM 14

Software Configuration Management – Installation and Configuration using Ansible

Aim:

To install and configure a Software Configuration Management (SCM) tool (Ansible) and use it to automate configuration tasks on managed nodes.

Objectives:

1. To understand the need for configuration management in DevOps.
2. To install and configure Ansible on a control/master machine.
3. To create an inventory file and establish SSH communication with managed nodes.
4. To write and execute a simple Ansible playbook for software installation/configuration.
5. To verify and validate the automated configuration changes.

Problem Statement:

Manual configuration leads to errors, delays and environment mismatches. Software Configuration Management (SCM) tools like Ansible automate this process by maintaining uniform configurations using scripts/playbooks. To install Ansible, configure it with an inventory and run a playbook to automatically install software on a target machine.

Procedure:

Step 1: Install Ansible

On Linux (Ubuntu/Debian):

sudo apt update

sudo apt install ansible -y

Check installation:

ansible –version

Step 2: Configure Inventory File

Create an inventory file: **sudo nano /etc/ansible/hosts**

Add managed node(s):

[webservers]

192.168.1.10

[database]

192.168.1.11

Step 3: Setup SSH Connection

On master/control machine:

ssh-keygen

ssh-copy-id user@192.168.1.10

Test connection:

ansible all -m ping

Expected output:

pong

Step 4: Create an Ansible Playbook

Create a playbook file named **install_apache.yml**:

```
- name: Install Apache Web Server
  hosts: webservers
  become: yes

  tasks:
    - name: Install Apache
      apt:
        name: apache2
        state: present

    - name: Start Apache service
      service:
        name: apache2
        state: started
```

Step 5: Run the Playbook

ansible-playbook install_apache.yml

Ansible automatically installs and starts Apache on remote nodes.

Step 6: Verify Output

On managed node browser or terminal:

curl http://192.168.1.10

You should see: Apache Default Page

Step 7: Check Ansible Logs

ansible webservers -m service -a "name=apache2 state=status"

Output:

Expected outputs:

Successful installation of Ansible:

- **ansible 2.x.x**

Successful SSH ping response:

- **192.168.1.10 | SUCCESS => { "ping": "pong" }**

Playbook execution summary:

- **PLAY RECAP**
- **192.168.1.10 : ok=2 changed=2 unreachable=0 failed=0**

Apache default webpage displayed through browser or curl.

Conclusion:

In this experiment, we installed and configured Ansible as a Software Configuration Management tool. It demonstrates how configuration management tools help achieve consistency, reduce errors, automate repetitive tasks and support efficient DevOps practices.

PROGRAM 15

Provisioning – To Perform Software Configuration Management and Provisioning using Chef / Puppet / Ansible / SaltStack

Aim:

To perform provisioning and software configuration management on remote servers using the Ansible automation tool.

Objectives:

1. To understand provisioning in DevOps.
2. To install and configure Ansible as a provisioning tool.
3. To create an inventory of managed nodes.
4. To write and execute an Ansible playbook to provision a server with required software.
5. To verify successful provisioning and configuration.

Problem Statement:

Provisioning involves automatically setting up servers by installing required software, configuring services and preparing the environment. Tools like Ansible automate provisioning and configuration management using scripts/playbooks, ensuring consistency across environments.

Procedure:

Step 1: Install Ansible on Control Node

For Ubuntu/Debian:

sudo apt update

sudo apt install ansible -y

Verify installation: **ansible --version**

Step 2: Configure Inventory File

Open the default inventory file:

sudo nano /etc/ansible/hosts

Add managed node(s):

[web]

192.168.1.15

Step 3: Set Up SSH Authentication (Control → Managed Node)

ssh-keygen

ssh-copy-id user@192.168.1.15

Test connection: **ansible all -m ping**

Expected response: **pong**

Step 4: Create Provisioning Playbook

Create a YAML file **provision_webserver.yml**:

```
---
- name: Provision Web Server
  hosts: web
  become: yes

  tasks:
    - name: Update package list
      apt:
        update_cache: yes

    - name: Install Apache Web Server
      apt:
        name: apache2
        state: present

    - name: Ensure Apache is running
      service:
        name: apache2
        state: started
        enabled: yes

    - name: Create a custom homepage
      copy:
        dest: /var/www/html/index.html
        content: "<h1>Provisioned Successfully using Ansible!</h1>"
```

Step 5: Run the Provisioning Playbook

```
ansible-playbook provision_webserver.yml
```

Step 6: Verify Successful Provisioning

On the managed node:

Command Line Check

curl http://192.168.1.15

Expected Output: <h1>Provisioned Successfully using Ansible!</h1>

Output:

Expected Terminal Output

- Successful playbook execution:

```
PLAY RECAP
192.168.1.15 : ok=4 changed=3 unreachable=0 failed=0
```

Conclusion:

We successfully performed provisioning and configuration management using Ansible. We installed Ansible, configured an inventory, established SSH communication and executed a playbook that automatically. This demonstrates how provisioning tools like Ansible automate server setup, ensure consistency and simplify DevOps workflows.